

Lab Assignment 3: How to Load, Convert, and Write JSON Files in Python

DS 6001

Problem 0

Make sure the virtual environment you created for this course is loaded as the notebooks kernel, and load the following packages:

```
In [50]: import numpy as np
import pandas as pd
import requests
import json
```

Problem 1

JSON and CSV are both text-based formats for the storage of data. It's possible to open either one in a plain text editor. Given this similarity, why does a CSV file usually take less memory than a JSON formatted file for the same data? Under what conditions could a JSON file be smaller in memory than a CSV file for the same data? [10 points]

Answer:

JSON is a tree based structure, while CSV is tabular. JSON works with key value pairs, where often the same key represents a column in the data structure. This creates a lot of repetition when we use the same type of object to represent multiple items. On the other hand, CSV can represent objects as records under the same column names, which makes it relatively smaller.

However, because CSV needs to maintain its tabular shape while JSON is more flexible, JSON can be smaller in some cases. JSON can model a tree using only the nodes that are needed, while CSV needs to model the data in a rectangular structure, including cells where values may be missing.

Problem 2

A [User-Agent header](#) (also called a user-agent string) is text that identifies the software you are using to access data from a web-server. It is considered to be good etiquette to write your user-agent string and send it to a web server along with your data request,

and some web servers will enforce that by refusing to send you data unless you provide a user-agent string.

Like a lot of things in software, there are stringent conventions that people follow when it comes to user-agent strings, and to operate in this world, you need to know how to abide by these conventions.

But sometimes conventions don't make a lot of sense. This question will show you some of the ways that user agents are understood by systems on the internet, and also why these systems have gotten more confusing over time.

Part a

Use your browser (Chrome, Firefox, Safari, etc.) to view the content at <https://httpbin.org/user-agent>. (If this website is down, you can accomplish the same thing by going to Google and searching for "what is my user agent?") Copy the user-agent string and paste it here. Then identify the parts of this user-agent string that identify your web browser and your computer's operating system (Mac, Windows, etc.) [6 points]

Answer:

```
{ "user-agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36" }
```

OS: Macintosh; Intel Mac OS X 10_15_7

Browser: Chrome/152.0.0.0

Part b

Which parts of the user agent you found in part (a) appear to be incorrect, or referring to a system other than your own? Then read this [blog post](#) and provide a 1-2 sentence explanation for why "Mozilla/5.0" probably appears in your user agent string. [6 points]

Answer:

Mozilla/5.0 and Safari/537.36 appear in my user-agent string even though I am not using either Mozilla or Safari. After reading the article, I learned that Mozilla/5.0 appears because many browsers began identifying themselves as other browsers to gain access to websites and features that otherwise might not support them.

Part c

The best way to provide a transparent and honest user agent string is to use the format required by Wikipedia. Wikipedia's [documentation for user-agent policy](#) requires us to

use a model such as `CoolBot/0.0` (`https://example.org/coolbot/; coolbot@example.org`) `generic-library/0.0`. The problem is that we aren't building software at the moment, so it's not clear what we should write for the bot name, version number, and website. I think the most honest thing we can do is refer to the exact application we have in mind, and supply a short written description instead of website. Fill out the information in the following code block to create a properly-formatted user agent, stored in a dictionary named `myheaders` (which we'll use later in this lab.) [2 points]

```
In [51]: mybot = 'ds6001_lab3_student_006' # you can choose a different name if you w
botversion = '0.0'
botwebsite = 'learning JSON' #write a short description of what you are tryi
email = 'qrk9cs@virginia.edu' #your email
software = 'python-requests'
softwareversion = requests.__version__

useragent = f'{mybot}/{botversion} ({botwebsite}; {email}) {software}/{softw
myheaders = {'User-Agent': useragent}
```

Part d

There is a huge industry built around the central task of pulling data off of websites. Most of this work is automated using specially-built software. Some software has been explicitly identified and banned from accessing certain websites due to misuse of the data or creating too much traffic for the web server to handle.

Websites often include a `robots.txt` file that lists the rules the web server asks users to follow, and publicizes the steps they've taken to restrict access. These files will sometimes call out specific user-agents. In the syntax of a `robots.txt` file, the phrase `Disallow: /` means "disallow this user from accessing anything on this website."

The following websites have `robots.txt` files that ban specific user-agents:

- <https://www.tiktok.com/robots.txt>
- <https://www.espn.com/robots.txt>
- <https://www.nytimes.com/robots.txt>
- <https://www.amazon.com/robots.txt>
- <https://www.cvilletomorrow.org/robots.txt> (You should check out <https://www.cvilletomorrow.org/>, an awesome local online newspaper with award-winning data journalism)

Choose one of these websites and find the specific user-agents that have been banned from access (user agents that have been named explicitly, not `User-Agent: *` which applies to all user agents), and copy-and-paste that part of the `robots.txt` file here. Then do an internet search for one of these agents. Based on what you learn about the

user-agent in question, why might you speculate that this user-agent was banned? (Any thoughtful answer to this part receives full credit). [6 points]

Answer:

User-agent: Amazonbot

Disallow: /wirecutter/

After searching, I found Amazon's documentation about Amazonbot, which mentions that the bot can be used to help develop and train Amazon's AI models. This makes me think that the New York Times may disallow this bot because they do not want their content to be collected and used to train AI models. This could help protect authors' work or simply give the NY Times more control over how its content is used.

Problem 3

The NBA has saved data on all 30 teams' shooting statistics for the 2014-2015 season here: <https://stats.nba.com/js/data/sportvu/2015/shootingTeamData.json>. Take a moment and look at this JSON file in your web browser or with <https://jsonhero.io>. The structure of this particular JSON is complicated, but see if you can find the team-by-team data. In this problem our goal is to use `pd.json_normalize()` to get the data into a dataframe. The following questions will guide you towards this goal.

Part a

Download the raw text of the NBA JSON file using `requests.get()`, and provide your user-agent to the NBA API by setting the `header` argument to the `myheaders` dictionary you created in problem 2. Then use `json.loads()` (or the `.json` attribute of the `requests` output) to parse the dictionaries and lists in the JSON formatted data. [8 points]

```
In [52]: url = "https://stats.nba.com/js/data/sportvu/2015/shootingTeamData.json"
response = requests.get(url, headers=myheaders)
print(response)

#response.text
```

<Response [200]>

```
In [53]: nba_shooting_team_json = json.loads(response.text)
# nba_shooting_team_json
```

Part b

Based on your observations of the JSON structure, describe in words the path that leads to the team-by-team data. [8 points]

Answer:

After looking at the data, the path that leads me to the team data starts with "resultSets". From there, we can see "headers", which contains the names of the columns, and "rowSet", which contains the information under those columns.

Part c

Use the `pd.json_normalize()` method to pull the team-by-team data into a dataframe.

[Note: what makes this tricky is that one of the layers in the path to the data is named only `0`. This `0` is not a string like the other keys. Specifying `0` in the `record_path`, either with or without quotes yields an error. There are two ways to solve this. The easiest way is to just skip over the `0` key entirely when writing down the `record_path`, and `pd.json_normalize()` will include the `0` layer on its own as a default behavior. The other way is to index the JSON data with both `['resultSets'][0]` first before passing the data to `pd.json_normalize()`, then writing the remaining layers in `record_path`]

If you are successful, you will have a dataframe with 30 rows and 33 columns. The first row will refer to the Golden State Warriors, the second row will refer to the San Antonio Spurs, and the third row will refer to the Cleveland Cavaliers. The columns will only be named 0, 1, 2, ... at this point. [10 points]

```
In [54]: nba_shooting_team_df = pd.json_normalize(nba_shooting_team_json, record_path='resultSets[0].headers')
print(nba_shooting_team_df.shape)
nba_shooting_team_df
```

(30, 33)

Out [54]:

	0	1	2	3	4	5	6	7	8	9	...
0	1610612744	Golden State	Warriors	GSW		82	48.7	114.9	14.9	0.498	...
1	1610612759	San Antonio	Spurs	SAS		82	48.3	103.5	14.8	0.481	...
2	1610612739	Cleveland	Cavaliers	CLE		82	48.7	104.3	16.9	0.481	...
3	1610612746	Los Angeles	Clippers	LAC		82	48.6	104.5	15.0	0.497	...
4	1610612760	Oklahoma City	Thunder	OKC		82	48.6	110.2	16.1	0.480	...
5	1610612737	Atlanta	Hawks	ATL		82	48.6	102.8	19.0	0.463	...
6	1610612745	Houston	Rockets	HOU		82	48.6	106.5	17.2	0.433	...
7	1610612757	Portland	Trail Blazers	POR		82	48.5	105.1	17.5	0.441	...
8	1610612758	Sacramento	Kings	SAC		81	48.4	106.7	18.7	0.452	...
9	1610612764	Washington	Wizards	WAS		82	48.5	104.1	15.4	0.480	...
10	1610612748	Miami	Heat	MIA		82	48.6	100.0	17.9	0.488	...
11	1610612761	Toronto	Raptors	TOR		81	48.5	102.7	23.0	0.462	...
12	1610612742	Dallas	Mavericks	DAL		82	49.0	102.3	18.2	0.473	...
13	1610612766	Charlotte	Hornets	CHA		82	48.6	103.4	16.8	0.459	...
14	1610612762	Utah	Jazz	UTA		82	49.0	97.7	18.1	0.445	...
15	1610612753	Orlando	Magic	ORL		81	48.7	102.0	18.0	0.456	...
16	1610612749	Milwaukee	Bucks	MIL		82	48.7	99.0	17.4	0.463	...
17	1610612740	New Orleans	Pelicans	NOP		82	48.5	102.7	19.9	0.458	...
18	1610612750	Minnesota	Timberwolves	MIN		82	48.6	102.4	15.1	0.464	...
19	1610612754	Indiana	Pacers	IND		82	48.8	102.2	13.7	0.453	...
20	1610612751	Brooklyn	Nets	BKN		82	48.4	98.6	14.4	0.457	...
21	1610612765	Detroit	Pistons	DET		82	48.7	102.0	17.5	0.464	...
22	1610612743	Denver	Nuggets	DEN		82	48.6	101.9	15.9	0.406	...
23	1610612738	Boston	Celtics	BOS		81	48.5	105.6	18.9	0.453	...
24	1610612741	Chicago	Bulls	CHI		82	48.9	101.6	18.1	0.458	...
25	1610612755	Philadelphia	76ers	PHI		82	48.6	97.4	19.7	0.445	...
26	1610612756	Phoenix	Suns	PHX		82	48.4	100.9	15.6	0.440	...
27	1610612752	New York	Knicks	NYK		82	48.5	98.4	10.4	0.447	...
28	1610612763	Memphis	Grizzlies	MEM		82	48.6	99.1	16.4	0.440	...

	0	1	2	3	4	5	6	7	8	9	...
29	1610612747	Los Angeles	Lakers	LAL		82	48.3	97.3	15.6	0.441	...

30 rows × 33 columns

Part d

Find the path that leads to the headers (the column names), and extract these names as a list. Then set the `.columns` attribute of the dataframe you created in part c equal to this list. The result should be that the dataframe now has the correct column names.

[Note: In this case, there's no need for `pd.json_normalize()` as the headers already exist as a list.]

[8 points]

```
In [55]: columns_name = nba_shooting_team_json["resultSets"][0]["headers"]
# columns_name
```

```
In [56]: nba_shooting_team_df.columns = columns_name
nba_shooting_team_df
```

Out [56]:

	TEAM_ID	TEAM_CITY	TEAM_NAME	TEAM_ABBREVIATION	TEAM_CODE	GP
0	1610612744	Golden State	Warriors	GSW		82
1	1610612759	San Antonio	Spurs	SAS		82
2	1610612739	Cleveland	Cavaliers	CLE		82
3	1610612746	Los Angeles	Clippers	LAC		82
4	1610612760	Oklahoma City	Thunder	OKC		82
5	1610612737	Atlanta	Hawks	ATL		82
6	1610612745	Houston	Rockets	HOU		82
7	1610612757	Portland	Trail Blazers	POR		82
8	1610612758	Sacramento	Kings	SAC		81
9	1610612764	Washington	Wizards	WAS		82
10	1610612748	Miami	Heat	MIA		82
11	1610612761	Toronto	Raptors	TOR		81
12	1610612742	Dallas	Mavericks	DAL		82
13	1610612766	Charlotte	Hornets	CHA		82
14	1610612762	Utah	Jazz	UTA		82
15	1610612753	Orlando	Magic	ORL		81
16	1610612749	Milwaukee	Bucks	MIL		82
17	1610612740	New Orleans	Pelicans	NOP		82
18	1610612750	Minnesota	Timberwolves	MIN		82
19	1610612754	Indiana	Pacers	IND		82
20	1610612751	Brooklyn	Nets	BKN		82
21	1610612765	Detroit	Pistons	DET		82
22	1610612743	Denver	Nuggets	DEN		82
23	1610612738	Boston	Celtics	BOS		81
24	1610612741	Chicago	Bulls	CHI		82
25	1610612755	Philadelphia	76ers	PHI		82
26	1610612756	Phoenix	Suns	PHX		82
27	1610612752	New York	Knicks	NYK		82
28	1610612763	Memphis	Grizzlies	MEM		82
29	1610612747	Los Angeles	Lakers	LAL		82

30 rows × 33 columns

Part e

Save the NBA dataframe you extracted in problem 4 as a JSON-formatted text file on your local machine. Format the JSON so that it is organized as dictionary with three lists: `columns` lists the column names, `index` lists the row names, and `data` is a list-of-lists of data points, one list for each row. [Hint: this is possible with one line of code] [8 points]

```
In [57]: new_json = nba_shooting_team_df.to_json(orient="split")
         #json.loads(new_json)
```

Problem 4

NASA has a public dataset of all asteroids that are in close proximity to Earth, searchable for any day, including today. The data contain the name of each asteroid, along with the absolute magnitude (a measure of luminosity), the minimum and maximum diameter in different units of measurement, the date and time of closest approach, the distance from Earth, and velocity of the asteroid at the moment of its closest approach.

To access the data for today, change the first line in the following code block to today's date in YYYY-MM-DD format:

```
In [58]: date = '2026-09-14'
         url = f'https://api.nasa.gov/neo/rest/v1/feed?start_date={date}&end_date={date}'
```

```
Out[58]: 'https://api.nasa.gov/neo/rest/v1/feed?start_date=2026-09-14&end_date=2026-09-14&api_key=DEMO_KEY'
```

(Note: APIs are systems for exchanging data between servers and users on the internet. We will be discussing APIs in depth in module 4. The URL above specifies `api_key=DEMO_KEY`, which is for initially exploring APIs prior to signing up. The demo key has more restrictive limits on the amount of data a user can acquire than you can get by signing up for your own key. For module 4 we will work through the process of getting our own API keys and keeping them secret while writing Python code, but for this exercise using the demo key will work for what we need.)

Use your web-browser or <https://jsonhero.io> to view the JSON data from this URL and determine the correct path that leads to the records we want to populate the rows of a dataframe.

Then use `requests.get()`, `json.loads()`, and `pd.json_normalize()` to bring the data into Python and organize it in a dataframe. Use the `headers=myheaders`

argument inside `requests.get()` to inform the NASA API about your user-agent string. [12 points]

```
In [59]: response = requests.get(url, headers=myheaders)
print(response)

#response.text
```

<Response [200]>

```
In [60]: asteroid_nasa_json = json.loads(response.text)
#asteroid_nasa_json
```

```
In [61]: asteroid_nasa_df = pd.json_normalize(asteroid_nasa_json, record_path=["near_asteroid_nasa_df
```

```
Out[61]:
```

	id	neo_reference_id	name	nasa_jpl_url	a
0	3031176	3031176	(2000 EB14)	https://ssd.jpl.nasa.gov/tools/sbdb_lookup.htm...	
1	3512704	3512704	(2010 FX9)	https://ssd.jpl.nasa.gov/tools/sbdb_lookup.htm...	
2	3612846	3612846	(2012 UD34)	https://ssd.jpl.nasa.gov/tools/sbdb_lookup.htm...	
3	3683469	3683469	(2014 QS295)	https://ssd.jpl.nasa.gov/tools/sbdb_lookup.htm...	
4	3835878	3835878	(2018 VG)	https://ssd.jpl.nasa.gov/tools/sbdb_lookup.htm...	

Problem 5

This problem used to be about pulling Reddit's top 25 posts on [/r/popular](#) into Python in JSON format. I had to scrap that because of the [API changes](#) Reddit announced in late 2025. That's a reminder that free and public data can be blocked and sold if that's what the data providers want to do. Keep this example in mind when you read the [article by Taina Bucher](#) for Module 4 next week.

Part a

Pull the data from Bluesky's API, with URL (and a couple parameters that Bluesky requires to use this API):

```
In [62]: url = 'https://public.api.bsky.app/xrpc/app.bsky.feed.getFeed'
params = {
    'feed': 'at://did:plc:z72i7hdynmk6r22z27h6tvur/app.bsky.feed.generator/w
    'limit': 50
}
```

Make sure to pass `myheaders` to the `headers` parameter of `requests.get()`, and to pass `params=params` as well. [2 points]

```
In [63]: response = requests.get(url, headers=myheaders, params=params)
reddit_top_post_json = json.loads(response.text)
#reddit_top_post_json
```

Part b

Look at the JSON output, and find the top listing (JUST the top listing, no others just yet). From this listing, write code that extracts the following (each in a different code cell):

- The handle for that post's author
- The 'createdAt' time for the post (Not the 'createdAt' time for the user's account)
- The text for the post
- The 'feedContext' (another way to say the topic) for the post

[8 points]

```
In [64]: author = reddit_top_post_json['feed'][0]['post']['author']['handle']
createdAt = reddit_top_post_json['feed'][0]['post']['record']['createdAt']
text = reddit_top_post_json['feed'][0]['post']['record']['text']
feedContext = reddit_top_post_json['feed'][0]['feedContext']

print(author)
print(createdAt)
print(text)
print(feedContext)
```

```
michaelwarburton.bsky.social
2026-09-14T19:43:47.561Z
```

```
Cher's outfit reveals on The Cher Show (1975) designed by BOB MACKIE – who
passed away today at the age of 87.
th-music
```

Part c

Using [the relevant section of the class textbook](#) as a guide, create a loop that pulls the four pieces of information from part (b) for all of the posts in the JSON data. [6 points]

```
In [65]: reddit_top_post_df = pd.DataFrame(
    [
        key['post']['author']['handle'],
        key['post']['record']['createdAt'],
        key['post']['record']['text'],
        key['feedContext'],
```

```
    ] for key in reddit_top_post_json['feed']  
)  
  
reddit_top_post_df.columns = ['author', 'createdAt', 'text', 'feedContext']  
reddit_top_post_df
```

Out [65]:

	author	createdAt	text
0	michaelwarburton.bsky.social	2026-09-14T19:43:47.561Z	Cher's outfit reveals on The Cher Show (1975) ...
1	cultmtl.com	2026-09-14T19:20:07.481Z	"The IPCC published a special report in 2018 s...
2	wajali.bsky.social	2026-09-14T19:59:54.806Z	Macklemore cemented his legacy. He will be res...
3	jennifershin.bsky.social	2026-09-14T11:07:46.331Z	Mother Nature has a solution.....
4	pmamtraveller.bsky.social	2026-09-14T13:45:52.933Z	The New American Gothic, by Criselda Vasquez
5	womensartbluesky.bsky.social	2026-09-14T19:09:04.516Z	The work of Diana Catchpole, contemporary UK b...
6	thegodshow.com	2026-09-14T20:48:20.290Z	Did Mitch show up today or is he still dead
7	jefftiedrich.bsky.social	2026-09-14T14:52:32.127Z	here's today's post: "why the fuck were millio...
8	hannibal.bsky.social	2026-09-14T19:32:16.401Z	Este vídeo es del telediario de TVE en 1990.\n...
9	thedailyshow.com	2026-09-14T17:59:08.314Z	Supermom @desilydic.bsky.social makes packing ...
10	pmamtraveller.bsky.social	2026-09-14T18:15:25.981Z	The Pearl With The Girl Earring, by Olaf Falafel
11	minakimes.bsky.social	2026-09-14T18:43:48.537Z	NFL Live from Kansas City! 3-5 pm ET on ESPN
12	eldiario.es	2026-09-14T15:56:18.600Z	"La vergüenza de Oviedo": David Uclés pide la ...
13	theserfstv.bsky.social	2026-09-14T18:52:12.307Z	Macklemore "every child deserves to live"\n\nE...
14	maxima-infinity.bsky.social	2026-09-14T06:17:00.156Z	
15	flbe.bsky.social	2026-09-14T16:10:11.556Z	Yesterday's harvest
16	elzo.eurosky.social	2026-09-14T14:24:02.128Z	12 años llevaba un ciclista español sin ganar ...
17	myandra.bsky.social	2026-09-14T17:28:36.086Z	Hearty little snowbirds. I love how they gathe...
18	anarchymarshmallow.bsky.social	2026-09-14T17:14:58.079Z	Ed Sheeran could have refused to boot Macklemo...
19	jojomineo69.bsky.social	2026-09-14T17:47:45.690Z	A fond farewell to the GOAT of Hollywood costu...

	author	createdAt	text
20	ribeirotrilhas.bsky.social	2026-09-14T17:33:40.112Z	Pesquisa DATA POVO.
21	ditzkoff.bsky.social	2026-09-14T16:42:54.616Z	RIP Bob Mackie. www.nytimes.com/2026/09/14/s...
22	goldengateblond.bsky.social	2026-09-14T18:16:30.215Z	he did many of cher's and dolly's most glamoro...
23	eldiario.es	2026-09-14T17:47:40.935Z	El discurso en defensa de Palestina de Macklem...
24	diedrichbader.bsky.social	2026-09-14T20:16:31.383Z	When my son was 5 he had a sartorial flair\nTo...
25	spryoldlorax.bsky.social	2026-09-14T16:18:42.964Z	I've said it before and I'll say it again: Rea...
26	the-art-of-web.com	2026-09-14T19:47:11.734Z	"Goodnight moon" #synthart
27	uncleduke1969.bsky.social	2026-09-14T17:13:48.149Z	sometimes the goal of science is to help us be...
28	vera-kober-art.bsky.social	2026-09-14T19:09:47.236Z	My flower painting
29	trouw.nl	2026-09-14T20:03:40.345Z	Ruime steun voor vermogensbelasting, ook bij r...
30	parkroseperma.bsky.social	2026-09-14T19:10:58.185Z	I've seen a real uptick in self- sufficient pre...
31	dogs100.bsky.social	2026-09-14T17:53:03.410Z	Excuse me....
32	pmamtraveller.bsky.social	2026-09-14T18:47:46.265Z	By Chris Austin
33	vera-kober-art.bsky.social	2026-09-14T15:43:15.085Z	My flower art
34	forbes.com	2026-09-14T15:20:07Z	Macklemore defended his remarks last week, say...
35	publico.es	2026-09-14T14:35:57+00:00	Mítines multitudinarios, notable intención de ...
36	menestune.bsky.social	2026-09-14T19:58:20.178Z	This bear clearly loved this trough as these t...
37	taz.de	2026-09-14T13:15:17.883Z	Alle Gerichtsurteile zu den Kontrollen an den ...
38	dmissstear.bsky.social	2026-09-14T19:02:17.484Z	A few butterflies seen on walks in Ireland thi...
39	gatinarios.bsky.social	2026-09-14T18:56:56.689Z	a xícara de café que eu preciso pra começar es...

	author	createdAt	text
40	thebeeguy.bsky.social	2026-09-14T19:39:18.820Z	#theBeeAt3 \n\nBasic bee facts every day at 3p...
41	northsidepat.bsky.social	2026-09-14T16:50:39.442Z	Did I have a piece of cheesecake for lunch? Ye...
42	mrsbettybowers.bsky.social	2026-09-14T17:23:12.436Z	Rest in peace costume designer Bob Mackie (193...
43	womensartbluesky.bsky.social	2026-09-14T18:58:21.318Z	Sibylle von Olfers (1881-1916), German art tea...
44	mack2114.bsky.social	2026-09-14T15:03:46.002Z	Jeff Tiedrich, once again demonstrating his le...
45	mollyknight.bsky.social	2026-09-14T16:00:31.800Z	I just know Bob is designing rhinestone outfit...
46	eldiario.es	2026-09-14T10:08:14.989Z	!! Sánchez compartirá con Mamdani un acto sobre...
47	tewalsh340.bsky.social	2026-09-14T08:41:44.255Z	
48	climatecasino.net	2026-09-14T16:10:42.750Z	Your 'bonus moment of doom' for today:\n\n"it ...
49	latimes.com	2026-09-14T18:40:42.549Z	The decommissioned San Onofre nuclear power pl...